Introduction

In today's digital world, auto-complete systems have become an essential feature in search engines, messaging applications, and text editors. These systems enhance user experience by predicting the next word in a sentence, allowing for faster and more efficient communication. In this assignment, you will build a basic auto-complete system that suggests the most probable next word based on a given input.

Assignment Description

The core component of an auto-complete system is a language model, which assigns probabilities to word sequences. The goal is to determine the most likely next word based on previously typed words. For example:

> "I have a pen" is expected to have a higher probability than "I am a pen" since the first one seems to be a more natural sentence in the real world.

You can take advantage of this probability calculation to develop an auto-complete system. For instance, if a user types

> "I eat scrambled" Then you can find a word `x` such that "I eat scrambled x" receives the highest probability. If x = "eggs", the sentence would be "I eat scrambled eggs"

To achieve this, you will implement an **N-gram language model**—a fundamental approach used in natural language processing (NLP). N-gram models analyze the probability of word sequences based on their occurrence in a given dataset. This method is widely used in applications such as **machine translation and speech recognition**.

Run the following cell to load the packages you will need.

```
import math
import random
import numpy as np
import pandas as pd
```

Develop n-gram based Language Models

In this section, you will develop the n-grams language model.

- Assume the probability of the next word depends only on the previous n-gram.
- The previous n-gram is the series of the previous 'n' words.

The conditional probability for the word at position t in the sentence, given that the words preceding it are $w_{t-n} \cdots w_{t-2}, w_{t-1}$ is:

$$P(w_t \vee w_{t-n} \ldots w_{t-1})$$

You can estimate this probability by counting the occurrences of these series of words in the training data. The probability can be estimated as a ratio, where the numerator is the number of times word t appears after words $t-n$ through $t-1$ appear in the training data, and the denominator is the number of times word t-n through $t-1$ appears in the training data. Formally:

$$\hat{P}(w_t \vee w_{t-n} \ldots w_{t-1}) = \frac{C(w_{t-n} \ldots w_{t-1}, w_t)}{C(w_{t-n} \ldots w_{t-1})}$$

where:

- The function $C(\cdots)$ denotes the number of occurence of the given sequence.
- $\hat{P}$ means the estimation of P.
- Notice that denominator of the equation (2) is the number of occurence of the previous n words, and the numerator is the same sequence followed by the word w_t.

Later, you will modify the equation (2) by adding k-smoothing, which avoids errors when any counts are zero.

The equation (2) tells us that to estimate probabilities based on n-grams, you need the counts of n-grams (for denominator) and (n+1)-grams (for numerator).

Task 1: Implementing n-Gram Count Computation

Next, you will implement a function that computes the counts of n-grams for an arbitrary number n.

Sentence Preparation:

Before computing the counts for n-grams, prepare the sentence by:

1. **Prepending $n-1$ start markers <s>** to indicate the beginning of the sentence.
   - For example, in a tri-gram model ($n=3$), a sequence with two start tokens (`<s> <s>`) should predict the first word of the sentence.
   - If the sentence is `"I like food"`, modify it to: `"<s> <s> I like food"`
2. **Appending an end marker <e>** so that the model can learn when to stop generating a sentence.

Technical Notes:

- The counts will be stored in a **dictionary**.
- Each **key** in the dictionary is a **tuple** of n words (not a list).
- Each **value** represents the number of occurrences of the corresponding n-gram.

Why Use a Tuple Instead of a List?

- In Python, **lists are mutable**, meaning they can be changed after creation.
- **Tuples are immutable**, making them a suitable data type for dictionary keys since they cannot be modified after creation.

Additional Note

Although an n-gram model typically uses $n-1$ starting markers, for this implementation, you should prepend n **start markers**.

- This ensures that later in the assignment, you can use them to compute the initial probability for the $n+1$**-gram** model.

```
def count_n_grams(data, n, start_token='<s>', end_token = '<e>'):
    """
    Count all n-grams in the data

    Args:
        data: List of lists of words
        n: number of words in a sequence

    Returns:
        A dictionary that maps a tuple of n-words to its frequency
    """

    # Initialize dictionary of n-grams and their counts
    n_grams = {}

    ### START CODE HERE ###
    for sentence in data:
        sentence = [start_token] * n + sentence + [end_token]
        for i in range(len(sentence) - n + 1):
            ngram = tuple(sentence[i:i+n])
            n_grams[ngram] = n_grams.get(ngram, 0) + 1
    ### END CODE HERE ###

    # Return the dictionary of n-gram counts
    return n_grams

# test your code
sentences = [['i', 'like', 'a', 'cat'],
             ['this', 'dog', 'is', 'like', 'a', 'cat']]
unique_words = list(set(sentences[0] + sentences[1])) + ["<s>", "<e>",
"<unk>"]

print("Uni-gram:")
print(count_n_grams(sentences, 1))
print("Bi-gram:")
print(count_n_grams(sentences, 2))

assert count_n_grams(sentences, 1) == {('<s>',): 2, ('i',): 1,
```

```
('like',): 2, ('a',): 2, ('cat',): 2, ('<e>',): 2,
                                        ('this',): 1, ('dog',): 1,
('is',): 1}, "Test failed!"
print("Success!")
assert count_n_grams(sentences, 2) == {('<s>', '<s>'): 2, ('<s>',
'i'): 1, ('i', 'like'): 1, ('like', 'a'): 2,
                                        ('a', 'cat'): 2, ('cat',
'<e>'): 2, ('<s>', 'this'): 1, ('this', 'dog'): 1,
                                        ('dog', 'is'): 1, ('is',
'like'): 1}, "Test failed!"
print("Success!")
```

```
Uni-gram:
{('<s>',): 2, ('i',): 1, ('like',): 2, ('a',): 2, ('cat',): 2,
('<e>',): 2, ('this',): 1, ('dog',): 1, ('is',): 1}
Bi-gram:
{('<s>', '<s>'): 2, ('<s>', 'i'): 1, ('i', 'like'): 1, ('like', 'a'):
2, ('a', 'cat'): 2, ('cat', '<e>'): 2, ('<s>', 'this'): 1, ('this',
'dog'): 1, ('dog', 'is'): 1, ('is', 'like'): 1}
Success!
Success!
```

Task 2: Implementing the `estimate_probability` function

Next, estimate the probability of a word given the prior n words using the n-gram counts as follows:

$$\hat{P}(w_t \vee w_{t-n} \ldots w_{t-1}) = \frac{C(w_{t-n} \ldots w_{t-1}, w_t)}{C(w_{t-n} \ldots w_{t-1})}$$

This formula doesn't work when a count of an n-gram is zero..

- Suppose we encounter an n-gram that did not occur in the training data.
- Then, the equation (2) cannot be evaluated (it becomes zero divided by zero).

A way to handle zero counts is to adused Laplace-Smoothing as follows:

$$\hat{P}(w_t \vee w_{t-n} \ldots w_{t-1}) = \frac{C(w_{t-n} \ldots w_{t-1}, w_t) + 1}{C(w_{t-n} \ldots w_{t-1}) + |V|)}$$

For n-grams that have a zero count, the equation (3) becomes $\frac{1}{|V|)}$.

- This means that any n-gram with zero count has the same probability of $\frac{1}{|V|)}$.

Define a function that computes the probability estimate (3) from n-gram counts.

- The function takes in a dictionary n_gram_counts, where the key is the n-gram and the value is the count of that n-gram.
- The function also takes another dictionary n_plus1_gram_counts, which you'll use to find the count for the previous n-gram plus the current word.

```
def estimate_probability(word, previous_n_gram,
                         n_gram_counts, n_plus1_gram_counts,
vocabulary_size):
    """
    Estimate the probability of a next word using n-gram counts with
Laplace smoothing.

    Args:
        word (str): The next word to predict.
        previous_n_gram (list): A sequence of words of length n (n-
gram).
        n_gram_counts (dict): Dictionary containing counts of n-grams.
        n_plus1_gram_counts (dict): Dictionary containing counts of
(n+1)-grams.
        vocabulary_size (int): Total number of unique words in the
vocabulary.

    Returns:
        float: The estimated probability of the given word following
the previous n-gram.
    """

    # Convert the previous n-gram from a list to a tuple to use it as
a dictionary key
    previous_n_gram = tuple(previous_n_gram)
    ### START CODE HERE ###
    previous_n_gram_count = n_gram_counts.get(previous_n_gram, 0)
    n_plus1_gram = previous_n_gram + (word,)
    n_plus1_gram_count = n_plus1_gram_counts.get(n_plus1_gram, 0)
    probability = (n_plus1_gram_count + 1) / (previous_n_gram_count +
vocabulary_size)
    ### END CODE HERE ###

    return probability

# test your code
unigram_counts = count_n_grams(sentences, 1)
bigram_counts = count_n_grams(sentences, 2)
trigram_counts = count_n_grams(sentences, 3)

tmp_prob = estimate_probability("cat", ["a"], unigram_counts,
bigram_counts, len(unique_words))

print(f"The estimated probability of word 'cat' given the previous n-
gram 'a' is: {tmp_prob:.4f}")
```

```
assert estimate_probability("cat", ["a"], unigram_counts,
bigram_counts, len(unique_words)) == 3/12, "Test failed!"
print("Success!")
```

```
The estimated probability of word 'cat' given the previous n-gram 'a'
is: 0.2500
Success!
```

Estimate probabilities for all words

The function defined below loops over all words in vocabulary to calculate probabilities for all possible words.

- This function is provided for you.

```
def estimate_probabilities(previous_n_gram, n_gram_counts,
n_plus1_gram_counts,
                           vocabulary):
    """
    Estimate the probabilities of next words using n-gram counts with
laplace-smoothing.

    Args:
        previous_n_gram (list): A sequence of words of length n (n-
gram).
        n_gram_counts (dict): Dictionary containing counts of n-grams.
        n_plus1_gram_counts (dict): Dictionary containing counts of
(n+1)-grams.
        vocabulary (list): List of known words in the dataset.

    Returns:
        dict: A dictionary mapping each word in the vocabulary to its
estimated probability.
    """

    # Convert the previous n-gram from a list to a tuple to use it as
a dictionary key
    previous_n_gram = tuple(previous_n_gram)

    # Add the end token (<e>) and unknown token (<unk>) to the
vocabulary
    # Note: The start token (<s>) is not added since it should not
appear as the next word.
    vocabulary = vocabulary

    # Get the size of the vocabulary (total unique words including
special tokens)
    vocabulary_size = len(vocabulary)
    # Initialize an empty dictionary to store probabilities
    probabilities = {}
```

```
    # Iterate over every word in the vocabulary
    for word in vocabulary:
        # Compute the probability of this word being the next word
        probability = estimate_probability(word, previous_n_gram,
                                           n_gram_counts,
n_plus1_gram_counts,
                                           vocabulary_size)


        # Store the probability in the dictionary
        probabilities[word] = probability

    # Return the computed probability distribution over all possible
next words
    return probabilities

# Test your code
#bigrams
assert estimate_probabilities(["a"], unigram_counts, bigram_counts,
unique_words) == {'is':1/12,'i':1/12,

'cat':3/12,'this':1/12,

'a':1/12,'like':1/12,

'dog':1/12,'<s>':1/12,

'<e>':1/12, '<unk>':1/12}, "Test failed!"
print("Success!")
#trigrams
assert estimate_probabilities(["<s>", "<s>"], bigram_counts,
trigram_counts, unique_words) == {'is':1/12,'i':2/12,

'cat':1/12,'this':2/12,

'a':1/12,'like':1/12,

'dog':1/12,'<e>':1/12,

'<unk>':1/12, '<s>':3/12}, "Test failed!"
print("Success!")

Success!
Success!
```

Count and probability matrices

As we have seen so far, the n-gram counts computed above are sufficient for computing the probabilities of the next word.

- It can be more intuitive to present them as count or probability matrices.
- The functions defined in the next cells return count or probability matrices.
- This function is provided for you.

```
def make_count_matrix(n_plus1_gram_counts, vocabulary):
    """
    Create a count matrix for n-grams and their corresponding next-
word counts.

    Args:
        n_plus1_gram_counts (dict): A dictionary where keys are (n+1)-
grams (tuples)
                                        and values are their occurrence
counts.
        vocabulary (list): A list of unique words in the vocabulary.

    Returns:
        pd.DataFrame: A count matrix where rows correspond to n-grams,

                      columns correspond to vocabulary words,
                      and values represent occurrence counts.
    """

    # The start token (<s>) is excluded because it should not appear
as a next word
    col = [x for x in vocabulary if x != '<s>']

    # Extract unique n-grams from (n+1)-gram keys
    n_grams = []
    for n_plus1_gram in n_plus1_gram_counts.keys():
        n_gram = n_plus1_gram[:-1]  # Extract the first n words from
(n+1)-gram
        n_grams.append(n_gram)

    # Convert n-grams list to a set to remove duplicates, then back to
a list
    n_grams = list(set(n_grams))

    # Create a mapping from each unique n-gram to a row index in the
count matrix
    row_index = {n_gram: i for i, n_gram in enumerate(n_grams)}

    # Create a mapping from each word in the vocabulary to a column
index in the count matrix
    col_index = {word: j for j, word in enumerate(col)}

    # Define matrix dimensions
    nrow = len(n_grams)  # Number of unique n-grams
    ncol = len(col)  # Number of unique words in the vocabulary

    # Initialize the count matrix with zeros
```

```
    count_matrix = np.zeros((nrow, ncol))

    # Populate the count matrix using the (n+1)-gram counts
    for n_plus1_gram, count in n_plus1_gram_counts.items():
        n_gram = n_plus1_gram[:-1]  # Extract the first n words (n-
gram)
        word = n_plus1_gram[-1]  # Extract the last word (next word)
        if word == '<s>':
            continue
        # Ignore words that are not in the vocabulary
        if word not in vocabulary:
            continue

        # Get the row index corresponding to the n-gram
        i = row_index[n_gram]

        # Get the column index corresponding to the next word
        j = col_index[word]

        # Assign the count to the appropriate position in the matrix
        count_matrix[i, j] = count

    # Convert the count matrix into a Pandas DataFrame for better
readability
    count_matrix = pd.DataFrame(count_matrix, index=n_grams,
columns=col)

    return count_matrix

print('bigram counts')
display(make_count_matrix(bigram_counts, unique_words))
```

```
bigram counts
```

	dog	is	i	this	like	cat	a	<e>	<unk>
(like,)	0.0	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0
(cat,)	0.0	0.0	0.0	0.0	0.0	0.0	0.0	2.0	0.0
(dog,)	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
(i,)	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
(a,)	0.0	0.0	0.0	0.0	0.0	2.0	0.0	0.0	0.0
(is,)	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0
(<s>,)	0.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
(this,)	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Expected output

```
bigram counts
          cat    i   this   a  is   like  dog  <e>   <unk>
(<s>,)    0.0   1.0  1.0  0.0  0.0  0.0   0.0  0.0    0.0
(a,)      2.0   0.0  0.0  0.0  0.0  0.0   0.0  0.0    0.0
(this,)   0.0   0.0  0.0  0.0  0.0  0.0   1.0  0.0    0.0
```

```
(like,)   0.0   0.0  0.0  2.0  0.0  0.0   0.0  0.0    0.0
(dog,)    0.0   0.0  0.0  0.0  1.0  0.0   0.0  0.0    0.0
(cat,)    0.0   0.0  0.0  0.0  0.0  0.0   0.0  2.0    0.0
(is,)     0.0   0.0  0.0  0.0  0.0  1.0   0.0  0.0    0.0
(i,)      0.0   0.0  0.0  0.0  0.0  1.0   0.0  0.0    0.0

# Show trigram counts
print('\ntrigram counts')
trigram_counts = count_n_grams(sentences, 3)
display(make_count_matrix(trigram_counts, unique_words))
```

```
trigram counts

             dog   is    i  this  like  cat    a  <e>  <unk>
(like, a)    0.0  0.0  0.0   0.0   0.0  2.0  0.0  0.0    0.0
(dog, is)    0.0  0.0  0.0   0.0   1.0  0.0  0.0  0.0    0.0
(this, dog)  0.0  1.0  0.0   0.0   0.0  0.0  0.0  0.0    0.0
(<s>, <s>)   0.0  0.0  1.0   1.0   0.0  0.0  0.0  0.0    0.0
(<s>, this)  1.0  0.0  0.0   0.0   0.0  0.0  0.0  0.0    0.0
(is, like)   0.0  0.0  0.0   0.0   0.0  0.0  1.0  0.0    0.0
(a, cat)     0.0  0.0  0.0   0.0   0.0  0.0  0.0  2.0    0.0
(<s>, i)     0.0  0.0  0.0   0.0   1.0  0.0  0.0  0.0    0.0
(i, like)    0.0  0.0  0.0   0.0   0.0  0.0  1.0  0.0    0.0
```

Expected output

```
trigram counts
              cat    i   this   a  is   like  dog  <e>   <unk>
(dog, is)     0.0   0.0  0.0  0.0  0.0  1.0   0.0  0.0    0.0
(this, dog)   0.0   0.0  0.0  0.0  1.0  0.0   0.0  0.0    0.0
(a, cat)      0.0   0.0  0.0  0.0  0.0  0.0   0.0  2.0    0.0
(like, a)     2.0   0.0  0.0  0.0  0.0  0.0   0.0  0.0    0.0
(is, like)    0.0   0.0  0.0  1.0  0.0  0.0   0.0  0.0    0.0
(<s>, i)      0.0   0.0  0.0  0.0  0.0  1.0   0.0  0.0    0.0
(i, like)     0.0   0.0  0.0  1.0  0.0  0.0   0.0  0.0    0.0
(<s>, <s>)    0.0   1.0  1.0  0.0  0.0  0.0   0.0  0.0    0.0
(<s>, this)   0.0   0.0  0.0  0.0  0.0  0.0   1.0  0.0    0.0
```

The following function calculates the probabilities of each word given the previous n-gram, and stores this in matrix form.

- This function is provided for you.

```
def make_probability_matrix(n_plus1_gram_counts, vocabulary):
    count_matrix = make_count_matrix(n_plus1_gram_counts,
unique_words)
    b = (count_matrix.sum(axis=1)+len(vocabulary))
    a = count_matrix + 1
    prob_matrix = a.div(b, axis=0)
    return prob_matrix
```

```
print("bigram probabilities")
display(make_probability_matrix(bigram_counts, unique_words))
```

```
bigram probabilities

               dog        is         i      this      like       cat
a  \
(like,)  0.083333  0.083333  0.083333  0.083333  0.083333  0.083333
0.250000
(cat,)   0.083333  0.083333  0.083333  0.083333  0.083333  0.083333
0.083333
(dog,)   0.090909  0.181818  0.090909  0.090909  0.090909  0.090909
0.090909
(i,)     0.090909  0.090909  0.090909  0.090909  0.181818  0.090909
0.090909
(a,)     0.083333  0.083333  0.083333  0.083333  0.083333  0.250000
0.083333
(is,)    0.090909  0.090909  0.090909  0.090909  0.181818  0.090909
0.090909
(<s>,)   0.083333  0.083333  0.166667  0.166667  0.083333  0.083333
0.083333
(this,)  0.181818  0.090909  0.090909  0.090909  0.090909  0.090909
0.090909

               <e>     <unk>
(like,)  0.083333  0.083333
(cat,)   0.250000  0.083333
(dog,)   0.090909  0.090909
(i,)     0.090909  0.090909
(a,)     0.083333  0.083333
(is,)    0.090909  0.090909
(<s>,)   0.083333  0.083333
(this,)  0.090909  0.090909
```

```
print("trigram probabilities")
trigram_counts = count_n_grams(sentences, 3)
display(make_probability_matrix(trigram_counts, unique_words))
```

```
trigram probabilities

                   dog        is         i      this      like
cat  \
(like, a)    0.083333  0.083333  0.083333  0.083333  0.083333
0.250000
(dog, is)    0.090909  0.090909  0.090909  0.090909  0.181818
0.090909
(this, dog)  0.090909  0.181818  0.090909  0.090909  0.090909
0.090909
(<s>, <s>)   0.083333  0.083333  0.166667  0.166667  0.083333
0.083333
```

```
(<s>, this)  0.181818  0.090909  0.090909  0.090909  0.090909
0.090909
(is, like)   0.090909  0.090909  0.090909  0.090909  0.090909
0.090909
(a, cat)     0.083333  0.083333  0.083333  0.083333  0.083333
0.083333
(<s>, i)     0.090909  0.090909  0.090909  0.090909  0.181818
0.090909
(i, like)    0.090909  0.090909  0.090909  0.090909  0.090909
0.090909

                     a       <e>     <unk>
(like, a)    0.083333  0.083333  0.083333
(dog, is)    0.090909  0.090909  0.090909
(this, dog)  0.090909  0.090909  0.090909
(<s>, <s>)   0.083333  0.083333  0.083333
(<s>, this)  0.090909  0.090909  0.090909
(is, like)   0.181818  0.090909  0.090909
(a, cat)     0.083333  0.250000  0.083333
(<s>, i)     0.090909  0.090909  0.090909
(i, like)    0.181818  0.090909  0.090909
```

Confirm that you obtain the same results as for the `estimate_probabilities` function that you implemented.

Task 3: Implementing the `calculate_perplexity` function

In this section, you will generate the **perplexity** score to evaluate your model on the test set.

- You will also use back-off when needed.
- Perplexity is used as an evaluation metric of your language model.
- To calculate the perplexity score of the test set on an n-gram model, use:

$$PP(W) = \sqrt[N]{\prod_{t=n+1}^{N} \frac{1}{P(w_t \vee w_{t-n} \cdots w_{t-1})}}$$

- where N is the length of the sentence.
- n is the number of words in the n-gram (e.g. 2 for a bigram).
- In math, the numbering starts at one and not zero.

In code, array indexing starts at zero, so the code will use ranges for t according to this formula:

$$PP(W) = \sqrt[N]{\prod_{t=n}^{N-1} \frac{1}{P(w_t \vee w_{t-n} \cdots w_{t-1})}}$$

The higher the probabilities are, the lower the perplexity will be.

- The more the n-grams tell us about the sentence, the lower the perplexity score will be.

Compute the perplexity score given an N-gram count matrix and a sentence:

```
def calculate_perplexity(sentence, n_gram_counts, n_plus1_gram_counts,
vocabulary_size, start_token='<s>', end_token = '<e>'):
    """
    Calculate perplexity for a list of sentences

    Args:
        sentence: List of strings (words in the sentence)
        n_gram_counts: Dictionary containing counts of n-grams
        n_plus1_gram_counts: Dictionary containing counts of (n+1)-
grams
        vocabulary_size: Total number of unique words in the
vocabulary

    Returns:
        Perplexity score (float)
    """
    perplexity = 0
    ### START CODE HERE ###
    n = len(list(n_gram_counts.keys())[0])
    sentence = [start_token] * n + sentence + [end_token]
    N = len(sentence)
    log_prob_sum = 0
    for t in range(n, N):
        previous_n_gram = sentence[t-n:t]
        word = sentence[t]
        prob = estimate_probability(word, previous_n_gram,
n_gram_counts, n_plus1_gram_counts, vocabulary_size)
        log_prob_sum += math.log2(prob)
    perplexity = 2 ** (-log_prob_sum / N)
    ### END CODE HERE ###
    return perplexity

# test your code
perplexity_train = calculate_perplexity(sentences[0],
                                        unigram_counts,
bigram_counts,
                                        len(unique_words))
print(f"Perplexity for first train sample: {perplexity_train:.4f}")

perplexity_test = calculate_perplexity(sentences[1],
                                       unigram_counts, bigram_counts,
                                       len(unique_words))
print(f"Perplexity for second sample: {perplexity_test:.4f}")
```

```
Perplexity for first train sample: 3.5819
Perplexity for second sample: 3.9873
```

Expected Output

```
Perplexity for first train sample: 3.5819
Perplexity for second sample: 3.9873
```

Note: If your sentence is really long, there will be underflow when multiplying many fractions.

- To handle longer sentences, you can modify your implementation to take the sum of the log of the probabilities.

Task 4: Build an Auto-complete System

In this section, you will combine the language models developed so far to implement an auto-complete system.

Compute probabilities for all possible next words and suggest the most likely one.

- This function also take an optional argument `start_with`, which specifies the first few letters of the next words.

```
def suggest_a_word(previous_tokens, n_gram_counts,
n_plus1_gram_counts, vocabulary, end_token='<e>',
unknown_token="<unk>", start_with=None):
    """
    Get suggestion for the next word based on n-gram probabilities.

    Args:
        previous_tokens: List of words representing the input
sentence. Must have length > n.
        n_gram_counts: Dictionary containing counts of n-grams.
        n_plus1_gram_counts: Dictionary containing counts of (n+1)-
grams.
        vocabulary: List of words in the vocabulary.
        end_token: Special token representing the end of a sentence.
        unknown_token: Token used for unknown words.
        start_with: If not None, specifies the first few letters of
the next word to filter suggestions.

    Returns:
        A tuple containing:
          - The most likely next word (string).
          - The corresponding probability (float).
    """

    # Determine the value of 'n' (size of n-grams) based on the keys
in n_gram_counts
```

```
    n = len(list(n_gram_counts.keys())[0])

    # Extract the most recent 'n' words from previous_tokens to form
the previous n-gram
    previous_n_gram = previous_tokens[-n:]

    # Compute probabilities for each word in the vocabulary as the
next word,
    # given the previous n-gram and the dictionaries of n-gram counts
    probabilities = estimate_probabilities(previous_n_gram,
                                           n_gram_counts,
n_plus1_gram_counts,
                                           vocabulary)

    # Initialize suggested word and maximum probability
    suggestion = None  # Will hold the best word suggestion
    max_prob = 0  # Will hold the highest probability encountered
    ### START CODE HERE ###
    for word, prob in probabilities.items():
        if start_with is not None and not word.startswith(start_with):
            continue
        if prob > max_prob:
            max_prob = prob
            suggestion = word
    ### END CODE HERE ###
    return suggestion, max_prob

# test your code
previous_tokens = ["i", "like"]
tmp_suggest1 = suggest_a_word(previous_tokens, unigram_counts,
bigram_counts, unique_words)
print(f"The previous words are 'i like',\n\tand the suggested word is
`{tmp_suggest1[0]}` with a probability of {tmp_suggest1[1]:.4f}")

print()
# test your code when setting the starts_with
tmp_starts_with = 'c'
tmp_suggest2 = suggest_a_word(previous_tokens, unigram_counts,
bigram_counts, unique_words, start_with=tmp_starts_with)
print(f"The previous words are 'i like', the suggestion must start
with `{tmp_starts_with}`\n\tand the suggested word is
`{tmp_suggest2[0]}` with a probability of {tmp_suggest2[1]:.4f}")
```

```
The previous words are 'i like',
     and the suggested word is `a` with a probability of 0.2500

The previous words are 'i like', the suggestion must start with `c`
     and the suggested word is `cat` with a probability of 0.0833
```

Expected output

```
The previous words are 'i like',
     and the suggested word is `a` with a probability of 0.2500

The previous words are 'i like', the suggestion must start with `c`
     and the suggested word is `cat` with a probability of 0.0833
```

Congratulations!

Congratulations on completing the assignment! Your dedication and effort are commendable. By successfully working through the coding exercises and written exercises, you have demonstrated a strong understanding of the concepts and principles related to regular expressions.

Congratulations on finishing this notebook!

Acknowledgement

About the Author

This notebook was authored by Mohamed Reda Bouadjenek. He is a Senior Lecturer (Assistant Professor) of Applied Artificial Intelligence in the School of Information Technology at Deakin University, Australia.

Contact Information

- **Email:** reda.bouadjenek@deakin.edu.au
- **GitHub:** https://github.com/rbouadjenek/